

Layer 4

Layer 4: The Transport Layer

Scope note: All of Layer 4 is examinable. Non-TCP/IP transport protocols (TP0–TP4, ATM, etc.) are explicitly **out of scope** per the lecturer's announcement. The protocols you must know are **UDP**, **TCP**, and **QUIC**.

1. Introduction — The "Pipe" Between Processes

The Transport Layer provides a **logical "pipe"** between two processes communicating across a network. At one end might be a web browser, at the other a web server; or an email program at one end and a mailbox server (IMAP4) at the other.

Two layers of hiding

The Transport Layer hides the **network's** complexity (routing, cable access, etc.) from the application-oriented layers, *and* it hides the application's behaviour (which process talks when, what data looks like) from the network-oriented layers. As far as the application is concerned, the network is just an abstract pipe; as far as the network is concerned, it is just providing a transport service.

Two core design decisions for any Layer 4 protocol

1. **How to identify the processes** at the endpoints of the pipe.
2. **How reliable the pipe should be.**

Monitoring transport-layer connections

MCQ fact: The command that enables one to see the status of transport layer connections on a host (in most operating systems) is `netstat`. (Not `ping`, not `tracert/traceroute`, not `tcp-show`, not `ps`.)

Reading netstat output

`netstat` (short for *network statistics*) displays active connections, listening ports, and associated timers. Example Linux output:

Local Address	Foreign Address	State	Timer
127.0.0.1:27117 (41.75/0/0)	127.0.0.1:48956	ESTABLISHED	keepalive
10.1.1.15:36870 (0.00/0/0)	10.1.3.169:8080	ESTABLISHED	off

```
127.0.0.1:6109      127.0.0.1:40310    TIME_WAIT    timewait
(10.00/0/0)
```

- **Local Address / Foreign Address:** the socket at each end (IP:port).
- **State:** the current TCP state (ESTABLISHED, TIME_WAIT, CLOSE_WAIT, etc.).
- **Timer column:** the currently active timer and the time remaining — `keepalive` means the keepalive timer is active on that idle connection; `off` means no timer; `timewait` means the quiet timer.

The `-r` flag (`netstat -r`) prints the routing table instead (Layer 3).

2. Identifying Processes — Ports and Sockets

Processes are identified by **numerical port numbers** carried in the Layer 4 header.

Port classifications

Range	Name	Use
0 – 1023	Well-known	Reserved for standard internet services. Require IANA registration to use.
1024 – 49151	Registered	Registered for specific applications but usable by others if open.
49152 – 65535	Dynamic / Private	Ad-hoc, requested by clients from the OS for receiving responses. Cannot be registered.

MCQ fact: Port **55555** is a **Dynamic** port (it falls in the 49152–65535 range).

Well-known ports table (memorise)

Port	Protocol
20	FTP (data)
21	FTP (control)
22	SSH
23	Telnet
25	SMTP
53	DNS
80	HTTP
110	POP3
143	IMAP

Port	Protocol
443	HTTPS

QUIC port note: QUIC currently uses **UDP port 443** for HTTPS (same number as TCP port 443). RFC 790 recommends that TCP and UDP port numbers be shared where possible, so applications transitioning to QUIC reuse the same port number.

Sockets

A **socket** is the combination of an IP address and a port: `<IP, port>`.

Example 1: When a web browser connects to `xx.co.za` on port 80, it opens the socket `<xx.co.za, 80>`.

Example 2: An SSH session from your laptop (`192.168.1.5:45231`) to a server (`10.0.0.1:22`) involves two sockets — the client socket `<192.168.1.5, 45231>` and the server socket `<10.0.0.1, 22>`.

Server-socket vs client-socket

A program opening a socket to act as a **server** must:

- Specify an **address** and **port number** (the well-known port its service runs on)
- **Wait for connections** (it does not initiate)

A client, conversely, *initiates* connections and obtains a dynamic port from the OS for its own side of the socket.

3. Reliability Mechanisms

A protocol is **reliable** if it guarantees data is delivered (or, if delivery fails, both ends know about the failure).

Connection-oriented vs connectionless

- **Reliable protocols** are typically **connection-oriented**: they explicitly establish, use, and tear down a connection. *Analogy*: a telephone call — you dial, talk, ask the other party to repeat unclear words, and hang up.
- **Unreliable protocols** are **connectionless**: no setup or teardown. *Analogy*: the snail-mail postal system — you send a letter blindly without knowing if the recipient is ready or even alive.

What can go wrong on a "pipe"

- Data **changes** before arriving (corruption)
- Data **never arrives** (loss)
- Data arrives **faster** than the receiver can cope (flow control needed)
- Data arrives **out of sequence**
- Data is **duplicated** and arrives multiple times

Flow control mechanisms

Flow control prevents a fast sender from overwhelming a slow receiver. The textbook identifies three mechanisms:

- **Stop-and-wait:** sender transmits one message, then halts completely until an ACK is received. Highly inefficient on long-latency links (both sender and receiver are idle most of the time).
- **Sliding window:** a specified number of messages (the *window*) may be sent without waiting for each individual ACK. Two variants:
 - **Go-Back-N:** if a message is lost, re-send the lost message *and every subsequent message* already sent.
 - **Selective Repeat:** re-send only the specifically lost message; all correctly received subsequent messages are kept.
- **Unrestricted:** no flow control at all. UDP uses this — it just sends without any restriction.

Example (sliding window): Window size = 4. A sends packets 1, 2, 3, 4. Packet 2 is lost. Go-Back-N would resend 2, 3, 4; Selective Repeat would resend only 2.

ARQ — Automatic Repeat reQuest

The term **ARQ** refers to flow control algorithms that use a timer: when a timer expires before an ACK arrives, the packet is retransmitted. Stop-and-wait and sliding window are both ARQ algorithms.

Minimum timer types for any sliding-window ARQ

A sliding-window ARQ protocol needs **at minimum 2 types of timers**:

1. A **retransmission timer** (at the sender): activated when a packet is sent; triggers retransmission if no ACK arrives in time.
2. An **acknowledgement timer** (at the receiver): activated when data arrives; triggers sending a standalone ACK if the data hasn't been acknowledged via a piggybacked ACK first.

MCQ fact: "What is the minimum number of timer types required by any sliding window ARQ protocol?" → **B: 2.**

TCP adds three more timers on top of these minimum two (persistence, keepalive, quiet — see §4.8).

Congestion control

A mechanism to **slow down transmission** when packet loss indicates the network is congested. Ensures fair sharing of bandwidth amongst all senders on the network.

4. TCP (Transmission Control Protocol)

TCP is the standard **reliable, connection-oriented** Layer 4 protocol. It uses a *selective repeat sliding window* that counts **bytes** rather than segments.

4.1 TCP reliability mechanisms (memorise!)

Exam Q (typical): "TCP is a reliable protocol. List any **five** mechanisms used by TCP to provide this reliability."

Any **five** of the following are accepted:

1. **3-way handshake to establish** the connection
2. **3-way handshake to terminate** the connection
3. **ACK for ARQ** (Automatic Repeat reQuest)
4. **Slow start** (congestion-window control)
5. **Exponential back-off** (on repeated loss)
6. **CRC / checksum over the entire segment**
7. **Window advertisements** (sliding-window flow control)
8. **Sequence numbers / acknowledgement numbers**

4.2 Connection establishment (3-way handshake)

The handshake exists because we have one path from A to B and another from B to A — both directions must be verified before reliable communication can be claimed.

```
A → B : SYN          (Are you there, ready to talk?)
A ← B : SYN+ACK       (Yes! Are *you* there?)
A → B : ACK           (Yes!)
```

Example: Your browser connecting to a web server. The browser sends SYN (port 80 on the server, a random high port on the client). The server replies SYN+ACK. The browser confirms with ACK. The connection is ESTABLISHED and the HTTP request can now be sent.

Phantom bytes — common MCQ trap

The SYN and SYN+ACK messages each consume **one sequence number** (the so-called *phantom byte*), even though they carry **no actual payload data**. The final ACK does **not**

consume a sequence number.

MCQ fact: "Which of the following statements is/are *false* about phantom bytes?"

- Counted as part of SYN ✓ (TRUE)
- Counted as part of SYN+ACK ✓ (TRUE)
- Counted as part of ACK ✗ (**FALSE**)
- Consists of one byte of data ✗ (**FALSE — no payload data**)

Answer: **More than one of the statements above are false** (C and D are both false).

Differentiating handshake packets (2024 ST2)

Q: "How are the SYN and ACK packets sent during a TCP handshake differentiated from one another?" → **A: The flags are used to determine the type of handshake packet.** ✓

Q: "How does the server differentiate between the ACK packet it receives as the last part of the handshake and an ACK packet that acknowledges data?" → **E: None of the above.**
(The server uses its current *TCP state* — it is in SYN-RECEIVED for the handshake ACK and ESTABLISHED for data ACKs.)

4.3 Connection termination

A separate 3-way (sometimes 4-way) handshake using FIN and FIN+ACK flags. Either party can initiate termination by sending FIN. After the final ACK, the connection enters TIME-WAIT.

Example: After a web page has been downloaded, the server sends FIN. The browser sends FIN+ACK (acknowledging the server's FIN and requesting its own close). The server sends ACK. The browser's side then enters TIME-WAIT before fully closing.

4.4 TCP states

All states defined verbatim from RFC 793:

State	Meaning
LISTEN	Waiting for a connection request from any remote TCP. (Acting as a server , waiting for SYN.)
SYN-SENT	Waiting for a matching connection request after having sent a SYN. (Client has sent SYN.)
SYN-RECEIVED	Waiting for a confirming ACK after having both received and sent a connection request.
ESTABLISHED	Open connection; data received can be delivered. The normal data-transfer state.

State	Meaning
FIN-WAIT-1	Waiting for a termination request from the remote TCP, or an ACK of the termination request sent.
FIN-WAIT-2	Waiting for a termination request from the remote TCP.
CLOSE-WAIT	Waiting for a termination request from the local user.
CLOSING	Waiting for a termination ACK from the remote TCP.
LAST-ACK	Waiting for an ACK of the termination request previously sent.
TIME-WAIT	Waiting for enough time to pass to be sure the remote TCP received the ACK of its termination request.
CLOSED	No connection state at all.

MCQ fact — LISTEN: "A TCP node that is in the LISTEN state:" → **A: Is acting as a server and waiting for a SYN message.** ✓

MCQ fact — TIME-WAIT: "A TCP connection in the TIME-WAIT state means that:" → **D: The node is waiting for 'lost' traffic to arrive at the port that is no longer in use, before it will be available for reuse.** ✓

Distractor: "The connection is about to time-out if no further data is transmitted soon" is **wrong**. TIME-WAIT is a deliberate waiting period after the connection closes, not a timeout warning.

4.5 TCP header fields

Standard TCP header (from RFC 793, as shown in the textbook):

```

0                               1                               2                               3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|                               |                               | | | | | | | | |
|      Source Port              |      Destination Port        |
|                               |                               |
|                               |      Sequence Number           |
|                               |                               |
|                               |      Acknowledgement Number    |
|                               |                               |
| Data |      |U|A|P|R|S|F|      |                               |
| Offset|  Rsv |R|C|S|S|Y|I|      |      Window                |
|      |      |G|K|H|T|N|N|      |                               |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|                               |                               |
|      Checksum                 |      Urgent Pointer           |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|                               |      Options                  |      Padding      |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+

```

Fields present: Source Port, Destination Port, Sequence Number, Acknowledgement Number, Data Offset (header length), Flags (URG, ACK, PSH, RST, SYN, FIN), **Window** (advertisement), Checksum, Urgent Pointer, Options, Padding.

MCQ fact: Which field does **NOT** occur in a TCP header? → **Length**. (Length is in the UDP header and in the IP header, but NOT in the TCP header. TCP derives length from the IP Total Length field.)

TCP flags

- **SYN** — synchronise (connection request / connection establishment)
- **ACK** — acknowledge (acknowledgement field is valid)
- **FIN** — finish (request to terminate)
- **RST** — reset (abort a connection immediately)
- **PSH** — push (deliver data immediately to the application, don't buffer)
- **URG** — urgent (Urgent Pointer field is valid; not discussed further)

4.6 The ACK flag and the acknowledgement field

The acknowledgement-number field is *only valid* when the **ACK flag is set**.

MCQ fact: "A sends a TCP segment to node B. The acknowledgement field contains the value 150. The ACK flag is **not** set. This means:" → **B: B should disregard the value 150 — it has no meaning.** ✓

Related fact — ACK's effect on timers: When node A sets the ACK flag in a segment to B, this may cause an existing **retransmission timer at A** to be **deleted** (because the data it was waiting to be acknowledged has now been acknowledged). The **acknowledgement timer at A** may also be deleted (the piggybacking opportunity was used). At B, the **acknowledgement timer** may be reset (a new acknowledgement obligation has arrived from A's data).

4.7 Damaged-packet handling — TCP has no NAK

TCP does **not** have a NAK (negative acknowledgement) flag.

MCQ fact: "A sends a TCP segment to B, but it arrives damaged. How is this resolved?" → **B: B discards the segment and does not acknowledge it; the retransmission timer at A expires and the packet is sent again.** ✓

4.8 TCP timers — memorise the five timers

Exam Q (asked almost every year): Very briefly state what happens when each TCP timer expires.

Timer	When activated	What happens when it expires
Retransmission timer	When a segment containing data is sent	A segment containing data is (re)transmitted . Timer restarted for the retransmitted segment.
Acknowledgement timer	When data arrives that needs acknowledging (delays ACK for piggybacking)	A segment containing an acknowledgement is transmitted (standalone ACK).
Persistence timer	When the sender is told the receiver's window has become 0	A probe message (a single non-data byte) is sent to check whether the receiver has freed up window space.
Keepalive timer	When a connection has been idle for a long period (~2 hours by default)	A probe is sent; if no response, connection resources are released .
Quiet timer	When the final FIN is sent (connection enters TIME-WAIT)	The port is freed and made available for reuse. Lasts twice the maximum segment lifetime.

Notes on each timer

Retransmission timer — RTT-based: The wait time is based on the **measured round-trip time (RTT_m)**: send a packet, measure how long the ACK takes. A smoothed weighted average RTT is maintained and used to set the timer.

Round-trip time (RTT): the total time for a message to travel from sender to receiver *and* the corresponding ACK to return.

Persistence timer — prevents deadlock: If A's last information is that B's window is 0, A stops sending. If B's window-update message is lost, A would wait forever (deadlock). The persistence timer prevents this by periodically probing B.

MCQ fact: "When the persistence timer at A expires, it implies that:" → **B: A is waiting for space to free up in B's window before further data can be sent.** ✓

Keepalive timer — detects stale connections: *Example:* A user SSHes into a server and goes to lunch. Their laptop catches fire and is destroyed. Without the keepalive timer the server would hold the connection open indefinitely. The keepalive timer detects that the client has disappeared and releases the connection.

Quiet timer and TIME-WAIT: Activated when the final FIN is sent in the termination handshake. Ensures any delayed segments from the now-closed connection cannot be mistaken for segments belonging to a new connection opened immediately after on the same port.

MCQ (2024 ST2): "The TCP *quiet* timer determines when a port may be used for a new connection. Which of the following is/are true?"

- A: The port is in the TIME-WAIT state while the quiet timer is active. ✓
- C: The quiet timer solves the problem of a packet from a previous connection being deemed part of a new connection. ✓ Answer: **D — More than one of the above.**

Definitions (Q4-style, 2024 ST2)

- **TCP slow start:** A congestion-control algorithm. The sender's congestion window (`cwnd`) starts at 1 segment. For each ACK received, `cwnd` increases — initially roughly exponentially. When loss is detected, `cwnd` is halved as a threshold and reset to 1; growth restarts linearly past the threshold.
- **TCP acknowledgement timer:** A timer that delays sending an ACK so it can be piggybacked onto reverse traffic. When it expires, a standalone ACK is transmitted.
- **Round-trip time (RTT):** The time for a message to travel from sender to receiver plus the time for the ACK to return. TCP measures it (`RTTm`) and smooths it to set the retransmission timer.
- **Middlebox (Layer 4 context):** A network device placed between TCP endpoints — typically a firewall or NAT — that "digs into" TCP headers (or payloads) to make decisions. Middleboxes violate the OSI layering ideal and are the primary reason TCP cannot easily be updated.
- **Protocol ossification:** The inability to modify a protocol because too many middleboxes have made assumptions about its on-the-wire form. Updating the protocol would break those middleboxes. QUIC was designed to combat this by encrypting headers and randomising version numbers (QUICv2 uses hex `6b3343cf`).

4.9 TCP flow control — sliding window in bytes

TCP's sliding window is **byte-oriented** (TCP numbers individual bytes, not segments). Each segment carries a **window advertisement** stating how much buffer space the receiver currently has available. The sender must not send more unacknowledged bytes than this window permits.

Example: B advertises a window of 1000. A has sent 600 unacknowledged bytes. A may send up to 400 more bytes before it must wait for an ACK.

4.10 TCP congestion control — slow start + exponential back-off

- **Congestion window (`cwnd`):** A second window kept at the *sender* (in addition to the receiver's advertised window). `cwnd` counts **segments** (not bytes).
- **Slow start:** Begin with `cwnd = 1` . For every ACK received, increase `cwnd` — growth is roughly exponential until a threshold or a loss occurs.
- **On loss:** Set the new threshold to `cwnd/2` , reset `cwnd` to 1, restart slow start. Once `cwnd` reaches the new threshold, switch to **linear growth** (avoidance phase) to avoid

overshooting.

Example: `cwnd` starts at 1, grows 1→2→4→8→16. Loss at 16. New threshold = 8, reset `cwnd` to 1. Slow start restarts: 1→2→4→8 (hits threshold) → 9→10→11 (linear). This is "slow start + exponential back-off."

4.11 Minimum number of timer types for any sliding-window ARQ

MCQ fact: "What is the minimum number of timer types required by any sliding window ARQ protocol?" → **B: 2** (retransmission timer + acknowledgement timer). Persistence, keepalive, and quiet are TCP-specific additions.

4.12 The TCP **Quiet** timer and TIME-WAIT (2024 ST2)

MCQ (2024 ST2 p): "The TCP *quiet* timer determines when a port may be used for a new TCP connection after a previous connection has been closed. Which claims are true?"

- A: The port (or socket) is in the TIME-WAIT state while the quiet timer is active. ✓
- B: The quiet timer is activated when the FIN+ACK message is sent. (Not quite right — it is activated at the third step of termination.)
- C: The quiet timer solves the problem of a packet from a previous connection being deemed part of a new connection. ✓

Answer: **D — More than one of the above** (A and C are both true).

5. UDP (User Datagram Protocol)

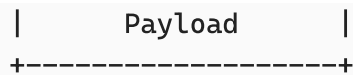
UDP is an **unreliable, connectionless** Layer 4 protocol offering a "best-effort" service.

5.1 What UDP lacks

- No flow control (described as **unrestricted** — meaning none at all)
- No congestion control
- No sequencing
- No acknowledgements
- No retransmission

5.2 UDP header (very simple — 4 fields only)

```
+-----+-----+
| Src Port| Dst Port|
+-----+-----+
| Length  | Checksum|
+-----+-----+
```



Fields: **Source Port**, **Destination Port**, **Length** (total datagram size including header), **Checksum**.

Key contrast with TCP: UDP *has* a **Length** field; TCP does **not** have a Length field (TCP derives length from the IP header). This is a frequent MCQ distractor.

5.3 When UDP is preferable

MCQ fact: Of Web page, File transfer, Audio, the data stream that generally works *better* with an unreliable Layer 4 protocol is **Audio**.

Reasons:

- **Real-time speech:** Dropping a fraction of a second of audio is preferable to halting playback to retransmit. Reliability introduces intolerable delays.
- **DNS lookup:** Only two messages (request + reply). A full TCP handshake + data + teardown = 8 messages for a 2-message transaction — wasteful overhead. DNS simply repeats the query if no answer arrives, possibly using a different server.

Example: A VoIP call uses UDP. A short gap in audio is barely noticeable; a retransmission-induced 500ms freeze would make conversation impossible.

5.4 DNS uses *both* TCP and UDP

MCQ fact: "Which of the following protocol(s) is/are used on the transport layer by DNS?" → **D: More than one of the above** (DNS uses **UDP** for standard queries and **TCP** for zone transfers and large responses).

6. QUIC (Quick UDP Internet Connections)

QUIC is a modern, **reliable, connection-oriented** Layer 4 protocol that runs **on top of UDP**. Google began work on it in 2012; QUICv1 was standardised in RFC 9000 (2022) and QUICv2 in RFC 9369 (December 2023).

6.1 Key facts

- QUIC provides reliability above unreliable UDP. The underlying UDP packets remain connectionless; QUIC supplies reliability on top.
- Informally called "TCP version 2". QUIC shares all TCP's goals but adds more.
- **Within ~10 years of its 2012 introduction, more than 50% of Web traffic was already QUIC.** Many predict QUIC will eventually replace TCP entirely.

- QUIC's original name was **Quick UDP Internet Connections** — three core properties: **quick, connection-oriented, on top of UDP**.
- Google's original version is **gQUIC**; the IETF-standardised version (RFC 9000/9369) is **IETF QUIC** (usually just "QUIC").

6.2 Security by default

QUIC inherently integrates **TLS 1.3** encryption. There is **no unencrypted version** of QUIC. The TLS handshake is bundled with the connection setup — not bolted on afterwards as with TCP+TLS.

Example: Wireshark can see that QUIC application data packets are being exchanged but cannot decrypt their contents, unlike TCP traffic without TLS.

6.3 Connection IDs and connection migration

TCP identifies a connection by the 4-tuple (`srcIP`, `srcPort`, `dstIP`, `dstPort`). If the client's IP changes (e.g. a mobile user switches from Wi-Fi to cellular data), TCP drops the connection. QUIC instead uses opaque **Connection IDs** selected by each endpoint.

MCQ fact: In QUICv1 and QUICv2, **the client** is the only party that can initiate a connection migration. (Not the server, not a network management system.) The server is explicitly prohibited from migrating in both QUICv1 and QUICv2.

How migration works: When the client's IP changes, it sends a packet from the new address using the same connection ID. The server verifies the client's identity using a cached token (a secret random number exchanged at connection setup). If valid, the server updates the client's address and continues.

Example: You stream a video over Wi-Fi. You walk out of range and your phone switches to LTE. TCP would drop the connection and the video app would rebuffer. QUIC seamlessly updates the server's record of your address using the connection ID, and playback continues without interruption.

6.4 Long-header fields always present

MCQ fact: Which headers are always present in a long QUIC header? (Include cases where the length of the field is present and may be 0, meaning the field itself may be omitted.)

- **Version**
- **Source connection ID**
- **Destination connection ID** Answer: **All of the above (E).**

Note on version numbers:

- QUICv1: version field = `00000001` (hex)

- QUICv2: version field = `6b3343cf` (hex) — deliberately arbitrary to prevent middleboxes from hardcoding pattern matching (ossification defence).

6.5 Speed — 1-RTT and 0-RTT

MCQ fact (memorise): "The claim that QUIC is a *quick* protocol means that:"

- **B:** It reduces latency when establishing or re-establishing a connection.
- **C:** Where multiple parts of a message must be transported, it ensures that parts are delivered quickly even if some are delayed (no head-of-line blocking between streams). Answer: **D — More than one of the above.**

TCP+TLS comparison

TCP+TLS requires 2 RTTs before application data can flow:

- 0 RTT: Client sends SYN → Server receives at t_1
- 1 RTT: Server sends SYN+ACK → Client receives at t_2
- 2 RTT: TLS handshake → Client can send data at t_4

QUIC reduces this:

- **1-RTT:** QUIC can send application data after 1 RTT (first connection).
- **0-RTT:** On a *reconnection* to a known server, the client can send encrypted application data in the **very first packet** using cached session keys from the previous connection.

The 0-RTT vs 1-RTT timing question

Assume a client establishes a new QUIC connection at time t_0 and the server replies at t_1 . The reply arrives at the client at t_2 . The client transmits at even times ($t_0, t_2, t_4, \dots$) and the server at odd times ($t_1, t_3, t_5, \dots$).

- **Soonest application data may (sometimes) be sent:** t_0 (via **0-RTT** using cached keys from a prior session).
- **Soonest 1-RTT application data may be sent:** t_2 (one full round trip after the initial t_0 send).
- **Soonest the connection is fully established:** t_3 (the server receives and processes the client's t_2 confirmation).

0-RTT risk — replay attacks: 0-RTT data may be replayed by an attacker (intercepting and re-sending the initial packet). Therefore, operations sent via 0-RTT must be **idempotent** (safe to repeat, like an HTTP GET). Non-idempotent actions (e.g. a bank transfer) must never use 0-RTT. HTTP defines status code **425 Too Early** to reject 0-RTT requests that are not safe to replay.

6.6 QUIC packet types and spaces

QUIC uses three distinct **packet-number spaces** — Initial, Handshake, and Application Data — each with their own independent numbering starting at 0. An ACK in an Initial packet can only acknowledge other Initial packets; it cannot acknowledge Handshake packets.

Packet types:

- **Initial** — early handshake, minimally encrypted (inspectable by Wireshark).
- **Handshake** — cryptographic handshake, better protected.
- **0-RTT** — early application data using cached keys.
- **1-RTT** — fully encrypted application data after handshake.

Only packets are acknowledged and retransmitted, not individual frames. If a packet is lost, QUIC creates a *new* packet (with a new, higher packet number) carrying the lost content.

6.7 Multiplexed streams — head-of-line blocking

Within a single QUIC connection, data is transported in multiple independent **streams**. If a packet on one stream is lost, only that stream is stalled while it retransmits; other streams continue unaffected. This avoids TCP's **head-of-line blocking** problem.

Example: A browser loads a webpage. It requests the HTML (stream 0), a CSS file (stream 4), an image (stream 8), and a script (stream 12) in parallel within one QUIC connection. If the image packet is lost, only stream 8 pauses to retransmit; the HTML, CSS, and script continue loading.

QUIC stream ID conventions

Stream IDs encode the initiator and directionality in their two lowest bits:

- Lowest bit 0 → **client-initiated**; bit 1 → **server-initiated**.
- Second-lowest bit 0 → **bidirectional**; bit 1 → **unidirectional**.

Example: A browser opens streams 0, 4, 8, 12 (all even = client-initiated, all second bit 0 = bidirectional). The server's stream-initiated responses are returned on the same stream IDs.

6.8 Frames

Packets carry **frames** as their payload. Common frame types:

- **STREAM** — carries application data. Contains stream ID, optional offset, optional length, and payload.
- **ACK** — acknowledges received packets (includes per-packet timestamps for accurate RTT measurement).
- **PING** — a keepalive; the receiver must acknowledge it, confirming the connection is alive.
- **CRYPTO** — carries TLS handshake data.

6.9 Protocol ossification — and QUIC's defences

TCP can't be upgraded because middleboxes (firewalls, NATs) have hardcoded assumptions about TCP headers and payloads — updates break those middleboxes.

QUIC defends against this in two ways:

1. **Heavy encryption of headers and metadata** — middleboxes cannot "dig into" most of the packet.
2. **Randomised version numbering** — QUICv2 uses `6b3343cf` (a deliberately arbitrary value). QUICv1 and QUICv2 also use *different* bit patterns to identify the same packet type, so a firewall hardcoded to recognise QUICv1 handshake packets by bit pattern won't recognise QUICv2's.

Example: A firewall might look for the TCP SYN bit pattern in Layer 4 packets to detect connection attempts. Because QUIC encrypts its headers, the firewall cannot make such assumptions — preventing the ossification that prevents TCP updates.

6.10 QUIC flow control

QUIC enforces flow control at two levels:

1. **Connection-level limit:** the total bytes of stream data across all streams.
2. **Stream-level limit:** the bytes allowed in any individual stream.

An endpoint that is blocked by a flow control limit sends a `DATA_BLOCKED` or `STREAM_DATA_BLOCKED` frame to notify the other party. These limits can be increased by the receiver sending `MAX_DATA` or `MAX_STREAM_DATA` frames.

6.11 QUIC port usage

QUIC is currently used primarily for HTTPS and commonly uses **UDP port 443** (same number as TCP port 443 for HTTPS). In practice, many browsers send simultaneous TCP and QUIC connection requests to port 443. If QUIC succeeds first, the TCP connection is abandoned.

Examinable Practice Questions on Layer 4

Multiple Choice

Q1. Which of the following data streams generally work(s) better with an unreliable Layer 4 protocol? A. Web page B. File transfer C. Audio D. More than one of the above E. All of the above

- Q2.** Which of the following protocol(s) is/are used on the transport layer by DNS? A. TCP B. UDP C. IP D. More than one of the above E. All of the above
- Q3.** Which command enables one to see the status of transport layer connections on a host (in most operating systems)? A. netstat B. ping C. tracert/traceroute D. tcp-show E. ps
- Q4.** UDP uses the following flow-control mechanism: A. Stop and Wait B. Sliding window C. Unrestricted D. More than one of the above E. All of the above
- Q5.** A sends a TCP segment to node B. The acknowledgement field contains the value 150. The ACK flag is *not* set. This means: A. B may assume that bytes up to byte 149 that it had sent are acknowledged. B. B should disregard the value 150 — it has no meaning. C. This is a negative acknowledgement of the segment B has sent that had the sequence number 150. D. The TCP layer at A is misconfigured; it should have set the ACK flag.
- Q6.** Which of the following fields does *not* occur in a TCP header? A. Source port B. Window advertisement C. Sequence number D. Options E. Length
- Q7.** Which of the following statements is/are *false* about phantom bytes used in an initial 3-way TCP/IP handshake? A. It is counted as part of the SYN message. B. It is counted as part of the SYN+ACK message. C. It is counted as part of the ACK message. D. It consists of one byte of data. E. More than one of the statements above are false.
- Q8.** A TCP connection in the `TIME-WAIT` state means that: A. The node is waiting for data from the node that it is connected to. B. The connection is about to time-out if no further data is transmitted soon. C. The connection will soon be established (as soon as the handshake is completed). D. The node is waiting for 'lost' traffic to arrive at the port that is no longer in use, before it will be available for reuse. E. The network is congested.
- Q9.** A TCP node that is in the `LISTEN` state: A. Is acting as a server and waiting for a SYN message. B. Is acting as a client and has sent a SYN message. C. May act as a client after sending a SYN message. D. More than one of the above. E. None of the above.
- Q10.** What does the claim that QUIC is a *quick* protocol mean? A. It manages the lower layers to transmit raw data at higher bit rates. B. It reduces latency when establishing or re-establishing a connection. C. Where multiple parts of a message have to be transported, it ensures that parts of the message are delivered quickly even if some parts are delayed. D. More than one of the above. E. All of the above.
- Q11.** QUIC connections can migrate. Which party can initiate such a migration in QUICv1 or QUICv2? A. The client B. The server C. The network management system D. More than one of the above E. All of the above
- Q12.** Say a damaged packet arrives at its destination. Suppose A sends a TCP segment to B, but it arrives damaged at B. How is this resolved? A. B sets the NAK TCP flag and sends it as a response to A. B. B discards the segment and does not acknowledge it; the retransmission timer at A expires and the packet is sent again. C. B requests the network

layer at B to deal with the error. D. B corrects the damaged segment and (eventually) acknowledges receipt. E. The lower layer protocols perform error checking, so a damaged segment will never arrive.

Q13. Suppose node A is connected to node B via TCP. When the persistence timer at A expires it implies that: A. B has not yet acknowledged the last segment A sent to B. B. A is waiting for space to free up in B's window before further data can be sent. C. A has no data to send, so it transmits a single byte to indicate that it is still present. D. More than one of the above.

Q14. What is the soonest time at which application data may (sometimes) be sent in QUIC? (Client at even times, server at odd times.) A. t_0 B. t_1 C. t_2 D. t_3

Q15. What is the soonest time at which **1-RTT** application data may be sent in QUIC? A. t_0 B. t_1 C. t_2 D. t_3

Q16. What type of port is port 55555? A. Registered B. Well-known C. Dynamic D. Practical E. Temporary

Q17. What is the minimum number of timer types required by any sliding window ARQ protocol? A. 1 B. 2 C. 3 D. 4 E. 5

Q18. A program that wants to open a socket in order to act as a server needs to specify: A. The address of the server B. The address of the client C. The port on the server D. The port on the client E. More than one of the above

Q19. Which of the following headers are always present in a *long* QUIC header? (Include cases where the length of the field will be present and may be 0, meaning the field itself may be omitted.) A. Version B. Source connection ID C. Destination connection ID D. More than one of the above E. All of the above

Q20. (2024 ST2) Which ISO OSI layer may provide checkpoints, such that only messages sent since the last checkpoint have to be retransmitted after a connection has been lost? A. 2 B. 3 C. 4 D. 5 E. 6

Q21. (2024 ST2) How are the SYN and ACK packets sent during a TCP handshake differentiated from one another? A. The flags are used to determine the type of handshake packet. B. These packets carry a phantom byte in which the type (SYN or ACK) is encoded. C. The sequence and acknowledgement numbers determine the type of packet. D. More than one of the above. E. All of the above.

Q22. Which of the following actions always happen(s) when a client establishes a QUIC connection with a server? A. The client chooses a connection ID to be used by the server. B. The client includes initial data encrypted with the server's public key. C. The client informs the server which cryptographic suite will be used. D. The client sends an initial packet numbered 0 on the wire. E. None of the above.

Q23. You see the following line in the output of `netstat` on Linux:

```
127.0.0.1:27117 127.0.0.1:48956 ESTABLISHED keepalive (41.75/0/0)
```

This connection: A. Is in use and actively exchanging data. B. Is in use, but idle; the keepalive timer is counting down. C. Is about to be closed due to a timeout. D. Is waiting for a SYN from the other party. E. Has been closed and is in TIME-WAIT.

Long-Form & Calculation

Q24. TCP timers definition. Very briefly state what happens when the following TCP timers expire: a) Retransmission timer b) Acknowledgement timer c) Persistence timer d) Keepalive timer e) Quiet timer

Q25. TCP reliability. TCP is a reliable protocol. List any **five** mechanisms used by TCP to provide this reliability. Keep your answers brief; no explanation required.

Q26. TCP flow control & sequence calculation (standard variant). Node A is busy communicating with node B via a TCP connection. At some time t the next byte A will send is byte 300. The next byte B will send is byte 500. The window size at A is 1000. The window size at B is 100.

- At time $t+1$ A wants to send **200** bytes to B. Call this message m_1 .
- At time $t+2$ B consumes 100 bytes from its buffer.
- At time $t+3$ B sends a message containing 50 bytes to A — message m_3 .
- At time $t+4$ A sends a message to B acknowledging m_3 (and including any residual data) — message m_4 .
- At time $t+5$ B sends an empty acknowledgement m_5 .

a) Sequence number included with m_1 ? b) Acknowledgement number included with m_4 ? c) Length of m_4 ? d) Window advertisement included with m_4 ? e) Window advertisement included with m_5 ?

Q27. TCP flow control & sequence calculation (2024 variant). Same setup as Q26 except A wants to send **300** bytes at $t+1$, and B consumes **200** bytes at $t+2$. Answer the same a)–e).

Q28. Definitions (2024 ST2 style). Briefly define or describe the following terms: a) TCP slow start b) TCP acknowledgement timer c) Round-trip time d) Middlebox (especially in the context of Layer 4) e) Protocol ossification

Q29. TCP header fields. The TCP header includes a flags field consisting of various flags. Name any **five other** fields that occur in the TCP header.

Memo / Answer Key

MCQ Answers

- A1.** C (Audio). Real-time media tolerates loss better than retransmission delay.
- A2.** D (More than one — DNS uses UDP normally and TCP for zone transfers).
- A3.** A (netstat).
- A4.** C (Unrestricted — UDP has no flow control).
- A5.** B (Disregard the value — without the ACK flag the field is meaningless).
- A6.** E (Length — present in UDP and IP headers, but NOT in TCP).
- A7.** E (More than one statement is false — C is false: phantom byte is *not* counted in the final ACK; D is false: phantom byte does *not* consist of payload data — it only consumes one sequence number).
- A8.** D (Waiting for lost traffic on the now-unused port before the port is reused).
- A9.** A (Acting as a server, waiting for a SYN).
- A10.** D (More than one of the above — both B and C are correct).
- A11.** A (The client — servers are explicitly prohibited from migrating in QUICv1 and QUICv2).
- A12.** B (TCP has no NAK; B discards the damaged segment and the retransmission timer drives recovery).
- A13.** B (Waiting for B's window to free up).
- A14.** A (t_0 — via **0-RTT** using cached keys from a previous session).
- A15.** C (t_2 — one full round trip after the initial t_0 send and the t_1 server reply).
- A16.** C (Dynamic — 55555 is in the 49152–65535 dynamic/private range).
- A17.** B (2 — a retransmission timer at the sender and an acknowledgement timer at the receiver).
- A18.** E (More than one — both an address and a port on the server must be specified, and the socket must wait for connections).
- A19.** E (All of the above — Version, Source connection ID, and Destination connection ID are always present in a long QUIC header).
- A20.** D (5 — Session Layer is responsible for checkpointing and rollback).

A21. A (The flags differentiate SYN from ACK from SYN+ACK).

A22. E (None of the above — the client does *not* choose the server's connection ID; the server picks its own. The client does not necessarily encrypt with the server's public key; QUIC uses a Diffie-Hellman-style exchange. The server selects the cipher suite from the client's offered list. QUIC does not number packets starting at 0 by convention — it may start at a higher random number).

A23. B (The connection is ESTABLISHED but idle; the `keepalive` timer is active and counts down. 41.75 seconds remain. This is not "about to close" — the keepalive timer runs for hours by default; 41 seconds remaining means it will reset on the next activity, or eventually probe if nothing arrives).

Long-Form Answers

A24. TCP Timers

- a) **Retransmission timer:** A segment containing data is (re)transmitted.
- b) **Acknowledgement timer:** A segment containing an acknowledgement is transmitted.
- c) **Persistence timer:** A probe message is sent (to check whether the receiver's window has freed up).
- d) **Keepalive timer:** Connection resources are released (or a probe is sent; if no answer, resources are released).
- e) **Quiet timer:** The port is made available for a new connection.

A25. TCP Reliability — any five of:

1. 3-way handshake to establish the connection
2. 3-way handshake to terminate the connection
3. ACK for ARQ
4. Slow start
5. Exponential back-off
6. CRC / checksum over the entire segment
7. Window advertisements (sliding-window flow control)
8. Sequence / acknowledgement numbers

A26. TCP flow-control & sequence calculation (200-byte variant)

Sub-question	Answer	Reasoning
a) Seq # of m_1	300	Next byte A sends is 300; m_1 starts at 300.
b) Ack # of m_4	550	B sent 50 bytes starting at 500 (bytes 500–549). A acknowledges next expected byte: 550.

Sub-question	Answer	Reasoning
c) Length of m_4	100	A wanted 200 but B's window allows only 100. m_1 sent 100 bytes; 100 residual bytes sent in m_4 .
d) Window advert in m_4	950	A's window = 1000. A received 50 bytes from B in m_3 (not yet consumed). A's free buffer = $1000 - 50 = 950$.
e) Window advert in m_5	0	B's window starts at 100. After m_1 (100 bytes), window = 0. B consumes 100, window back to 100. m_4 delivers 100 residual bytes, window returns to 0.

A27. TCP flow-control & sequence calculation (300-byte variant)

Sub-question	Answer	Reasoning
a) Seq # of m_1	300	Same — next byte to send is 300.
b) Ack # of m_4	550	Same — B sent 50 bytes from 500.
c) Length of m_4	200	A wanted 300 but B's window allowed only 100. Residual = 200. B consumed 200 bytes, freeing space; A sends all 200 residual bytes in m_4 .
d) Window advert in m_4	950	Same as above — A received 50 bytes from B, unconsumed.
e) Window advert in m_5	0	B's buffer accommodated 200 freed bytes; m_4 delivers 200 bytes, filling the space. Window = 0.

A28. Definitions

- a) **TCP slow start:** A congestion-control algorithm. `cwnd` starts at 1 segment. For each ACK, `cwnd` increases roughly exponentially (quickly probing capacity). When loss occurs, $\text{threshold} = \text{cwnd}/2$, `cwnd` = 1; growth restarts exponentially until the threshold, then linearly (combined = "slow start + exponential back-off").
- b) **TCP acknowledgement timer:** Started when data arrives; delays the ACK to allow piggybacking. When it expires, a standalone ACK is transmitted.
- c) **Round-trip time (RTT):** The total time for a message to travel from sender to receiver and the ACK to return. TCP measures RTTm and uses a smoothed average to set the retransmission timer.
- d) **Middlebox (Layer 4 context):** A device placed between TCP endpoints — typically a firewall or NAT — that inspects TCP headers (or payloads) to make decisions. Violates OSI layering and prevents TCP updates because any change would break the middlebox's hardcoded assumptions.
- e) **Protocol ossification:** The inability to modify a protocol because middleboxes and other equipment have made permanent assumptions about its wire format. Updating the

protocol breaks those assumptions. QUIC counteracts this by encrypting headers and using unpredictable version numbers (QUICv2 = 6b3343cf).

A29. TCP header fields (any five of): Source port, Destination port, Sequence number, Acknowledgement number, Data offset (header length), Window, Checksum, Urgent pointer, Options, Padding.